VERIQTA

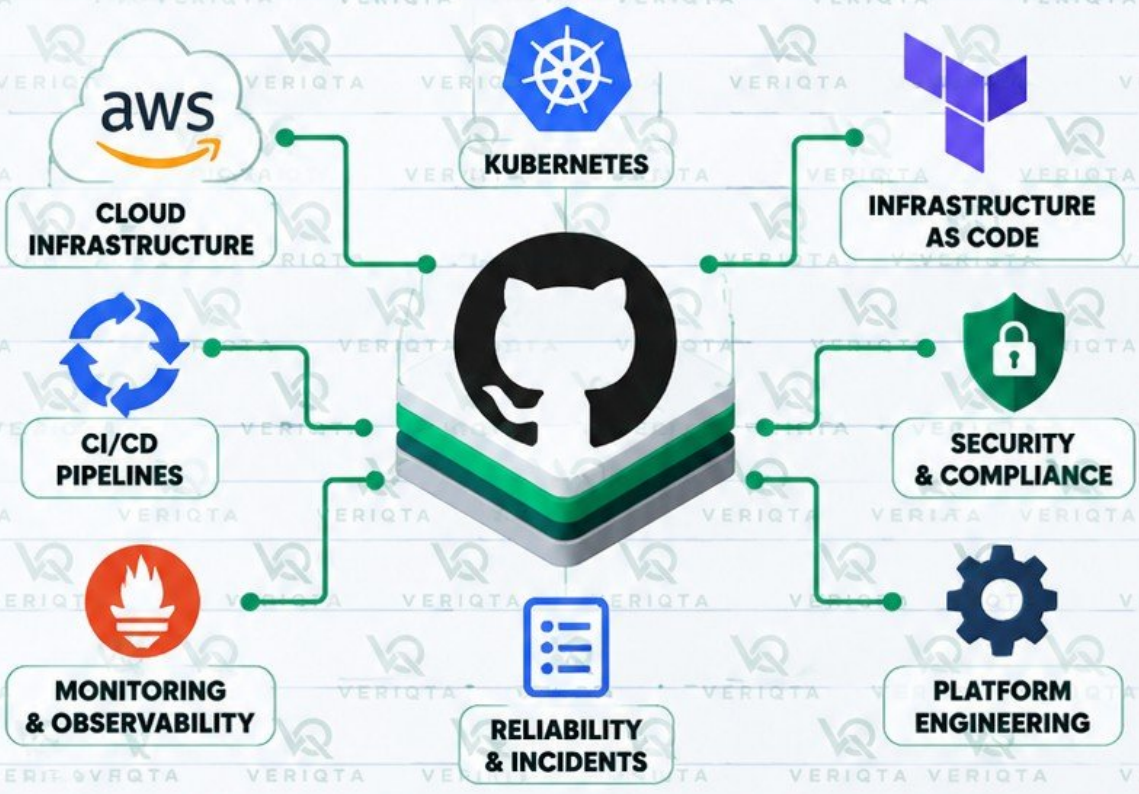
SENIOR ENGINEER

DevOps GitHub Portfolio Projects HANDBOOK

20 PROJECTS

Build • Automate • Secure • Scale • Operate

Real-World Projects for Senior DevOps Engineers



PRODUCTION READY PROJECTS | REAL-WORLD SCENARIOS | ARCHITECTURE DECISIONS | TROUBLESHOOTING & FAILURE TESTING | INTERVIEW PREPARATION

@veriqta @veriqta @veriqta @veriqta @veriqta @veriqta

GitHub Portfolio Deliverables

Every project must produce more than source code. Its repository should contain an architecture decision record, a reproducible deployment path, automated validation, a security and threat assessment, a controlled failure experiment, observability evidence, recovery documentation, and a clear explanation of cost and operational trade-offs.

Each project should also include:

01 — Production Incident Scenario
A realistic incident scenario with evidence and root-cause analysis.

02 — Working Implementation
A complete working implementation with verification and safe cleanup.

03 — Measured Results
Include measurable results, such as latency, availability, recovery time, or another relevant engineering metric.

04 — Documented Limitations
A clear explanation of limitations and what would need to change for real enterprise deployment.

05 — Complete GitHub README
Explain what was built, why the design was chosen, what failed during testing, and how the system recovered.

Important Distinction:
A reproducible lab can demonstrate production engineering practices, but it should not be presented as a live enterprise deployment unless it actually is one.

01 Enterprise Multi-Account AWS Landing Zone

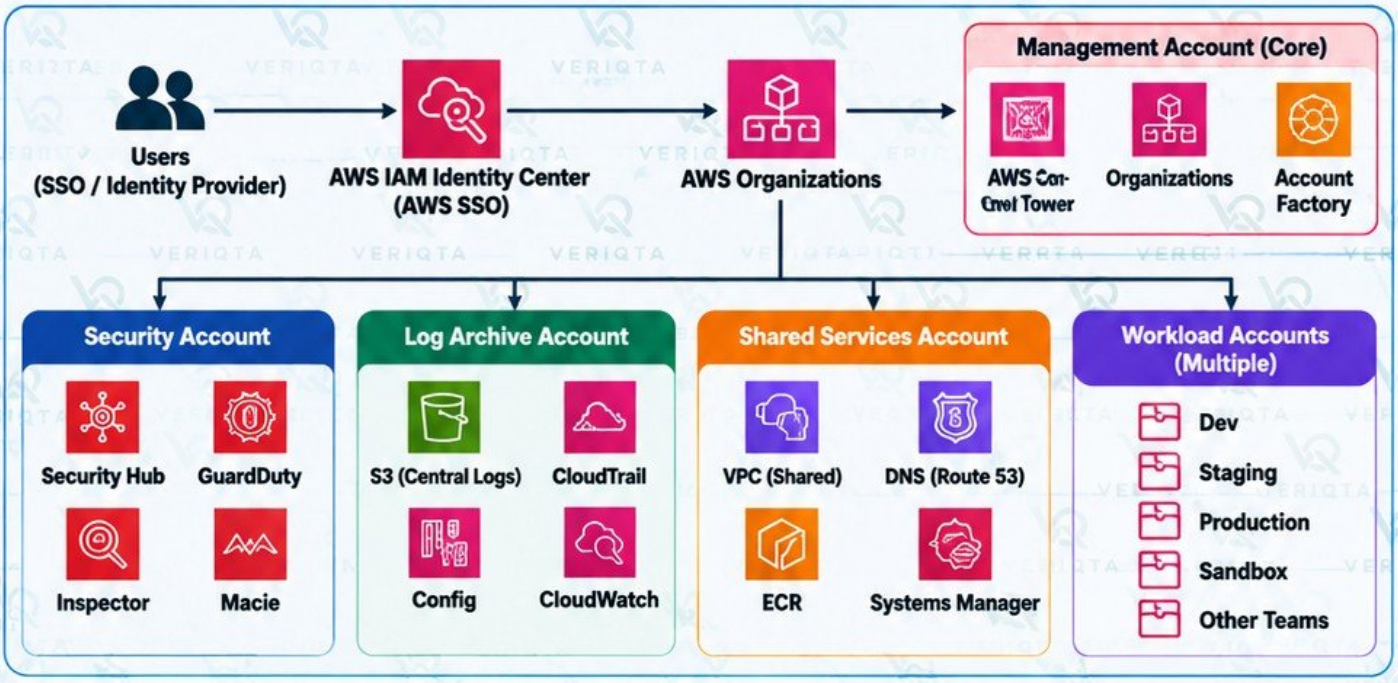
aws

Project Goal

Design a governed AWS foundation with account isolation, organizational controls, centralized audit logging, identity federation, and automated account provisioning.

Engineering Challenge

Prevent a compromised workload account from becoming an organization-wide incident.



Key Implementation Areas

- AWS Organizations structure and OUs
- AWS Control Tower for guardrails
- Account vending with automated provisioning
- IAM Identity Center (SSO) with federated identity
- Centralized logging and audit trail
- Baseline security and compliance controls
- Network architecture with shared services
- Account isolation and least privilege access

Tools and Services

- AWS Organizations
- AWS Control Tower
- IAM Identity Center
- AWS CloudTrail
- AWS Config
- Amazon S3
- AWS Security Hub
- Amazon GuardDuty

Success Criteria

- [x] New accounts provisioned automatically with guardrails
- [x] Centralized logging for all accounts
- [x] Federated access with least privilege
- [x] Verified account isolation (blast radius limited)
- [x] Security and compliance controls enforced

Real-World Impact

- [x] Stronger security posture
- [x] Prevent lateral movement across accounts
- [x] Full auditability and traceability
- [x] Faster, consistent account provisioning
- [x] Scalable foundation for enterprise workloads

02 Multi-Region Disaster Recovery Architecture

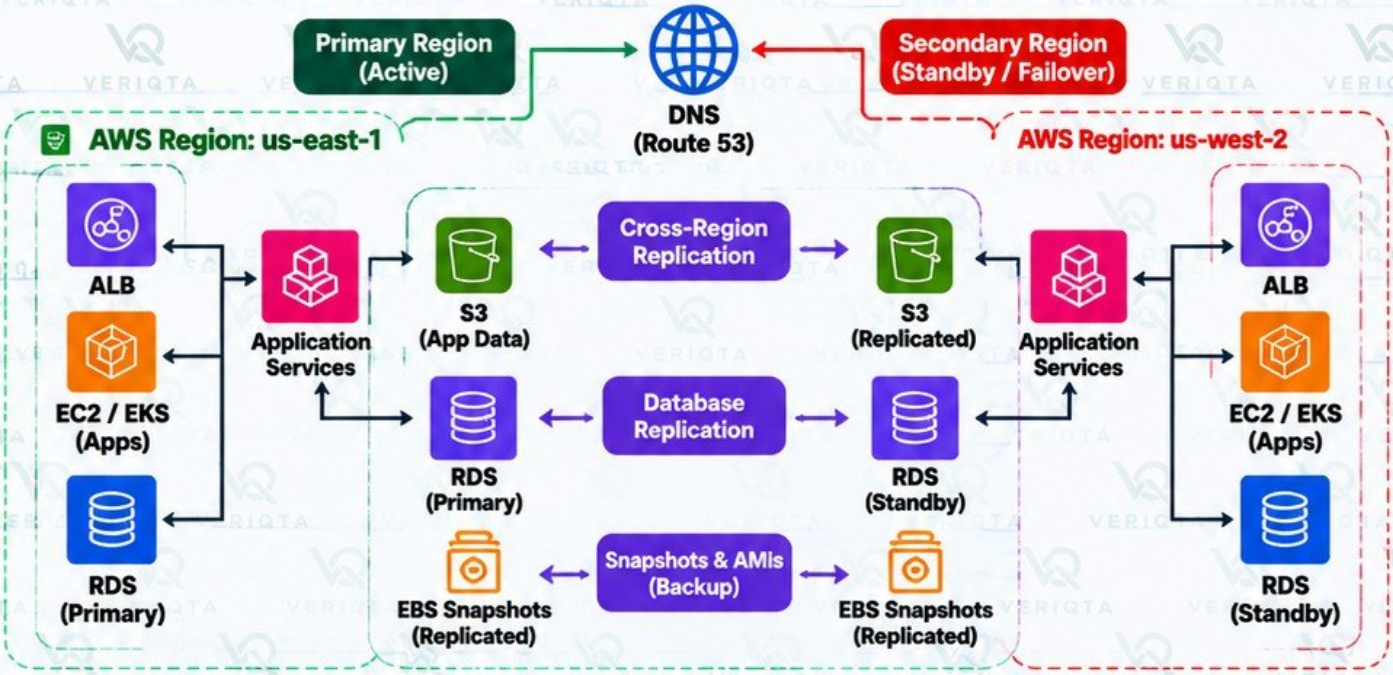
aws

Project Goal

Build a regional failover design with replicated application state, tested recovery procedures, DNS or traffic failover, and documented recovery objectives.

Engineering Challenge

Prove recovery time objective (RTO) and recovery point objective (RPO) through actual failure simulations.



Key Implementation Areas

- Multi-region network architecture (VPC, subnets, routing)
- Data replication (RDS, S3, EBS, application data)
- DNS or traffic failover (Route 53 health checks)
- Infrastructure as Code (Terraform)
- Automated backups and snapshot replication
- Recovery runbooks and automated failover scripts
- RTO and RPO definition and testing
- Monitoring, alerting and validation

Failure Scenarios to Test

- Complete region outage
- RDS primary failure
- Application tier failure
- Network connectivity loss
- DNS health check failure
- Data replication delay or failure
- Partial service degradation
- Rollback to primary region

Success Criteria

- [x] Achieve defined RTO (e.g., ≤ 1 hour)
- [x] Achieve defined RPO (e.g., ≤ 5 minutes)
- [x] Successful automated or manual failover
- [x] Data integrity verified after failover
- [x] Services accessible from new region
- [x] Recovery process documented and repeatable

Real-World Considerations

- [x] Cross-region data transfer cost
- [x] Latency impact for global users
- [x] Service limitations in each region
- [x] Application compatibility across regions
- [x] Testing regularly to avoid unnoticed drift
- [x] Clear ownership and failback strategy

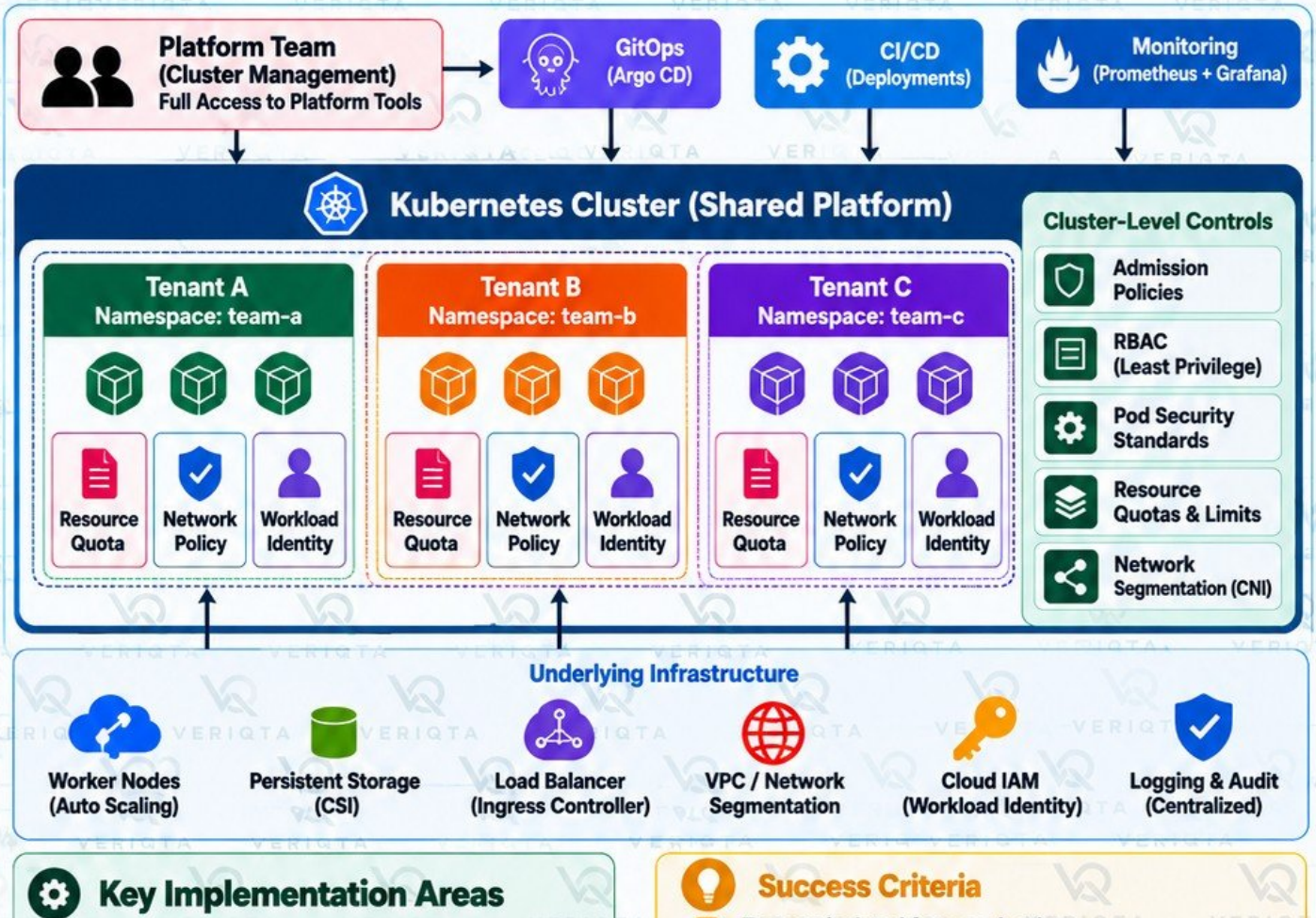
03 Production Kubernetes Platform with Multi-Tenant Isolation

Project Goal

Engineer a shared Kubernetes platform with workload isolation, admission policies, resource quotas, network segmentation, workload identity, and tenant-specific operational boundaries.

Engineering Challenge

Contain a compromised tenant and prevent resource starvation across teams.



Key Implementation Areas

- Namespace and tenant structure
- Resource quotas and limit ranges
- Network segmentation with NetworkPolicy
- Admission control (OPA / Kyverno)
- Workload identity (IRSA / OIDC)
- RBAC and team access boundaries
- Pod security standards and runtime security
- Observability per tenant (metrics, logs, traces)
- Automated provisioning for new tenants
- Chargeback and usage visibility (optional)

Success Criteria

- [x] Tenants isolated from each other
- [x] No resource starvation across teams
- [x] Policies enforced via admission control
- [x] Limited blast radius during compromise
- [x] Centralized observability and audit logs
- [x] Secure workload identity per tenant
- [x] Easy onboarding for new teams
- [x] Scalable and maintainable platform

Failure Scenarios to Test

- [x] Compromised pod in one tenant
- [x] Resource exhaustion attempt (CPU / Memory)
- [x] Network policy bypass attempt
- [x] Privilege escalation attempt
- [x] Malicious container image
- [x] Misconfigured RBAC or service account

04 Terraform Infrastructure Factory with Policy Enforcement

Project Goal

Build a reusable infrastructure delivery system with versioned modules, isolated state, drift detection, policy-as-code, automated testing, and controlled promotion.

Engineering Challenge

Detect and prevent destructive infrastructure changes before they reach production.

Infrastructure Delivery Pipeline

Develop Modules	Automated Validation	Policy as Code	Plan & Test	Approval Gate	Deploy (Environments)
• Reusable modules • Version control • Module registry	• Terraform fmt • terraform validate • tflint / checkov • Unit tests	• OPA / Sentinel • Policy checks • Block violations • Security & compliance	• terraform plan • Infracost (cost) • Automated tests • Ephemeral environment	• Manual/automated • Change review • Policy enforcement • Audit trail	• Dev → Staging → Production • Controlled promotion • Rollback ready

Terraform Module Structure

```
infrastructure-modules/
├── vpc/
├── eks/
├── rds/
├── iam/
├── modules.json
├── README.md
└── examples/
```

Environment Architecture

Dev Account	Staging Account	Production Account
VPC	VPC	VPC
EKS	EKS	EKS
RDS	RDS	RDS
State (Backend)	State (Backend)	State (Backend)

Remote State & Isolation

S3 (Remote State) + DynamoDB (State Locking)

- Separate state per environment
- Access control with IAM
- Encryption enabled
- Prevent concurrent modifications

Key Implementation Areas

- Modular infrastructure design and versioning
- Remote state with S3 and DynamoDB
- Automated validation and security scanning
- Policy enforcement (OPA/Sentinel)
- Environment separation and promotion workflow
- Drift detection and remediation
- Cost estimation and budget controls
- Automated testing with Terraform testing tools
- Audit logging and change tracking
- Rollback strategy for failed deployments

Success Criteria

- [x] Reusable and versioned modules
- [x] Policies block non-compliant changes
- [x] Drift detection identifies manual changes
- [x] Automated tests pass before deployment
- [x] Changes require approval for production
- [x] Clear audit trail of all changes
- [x] Cost impact visible before deployment
- [x] Secure and isolated state management
- [x] Successful deployment and rollback test
- [x] Repeatable process for new environments

Failure Scenarios to Test

- [x] Attempt to destroy a production VPC
- [x] Non-compliant resource (e.g. public S3)
- [x] Manual change causing drift
- [x] Policy bypass attempt
- [x] Excessive resource cost
- [x] Invalid module version
- [x] State lock contention
- [x] Failed deployment and rollback
- [x] Unauthorized access to state
- [x] Missing required tags or encryption

05 Zero-Trust CI/CD Software Supply Chain

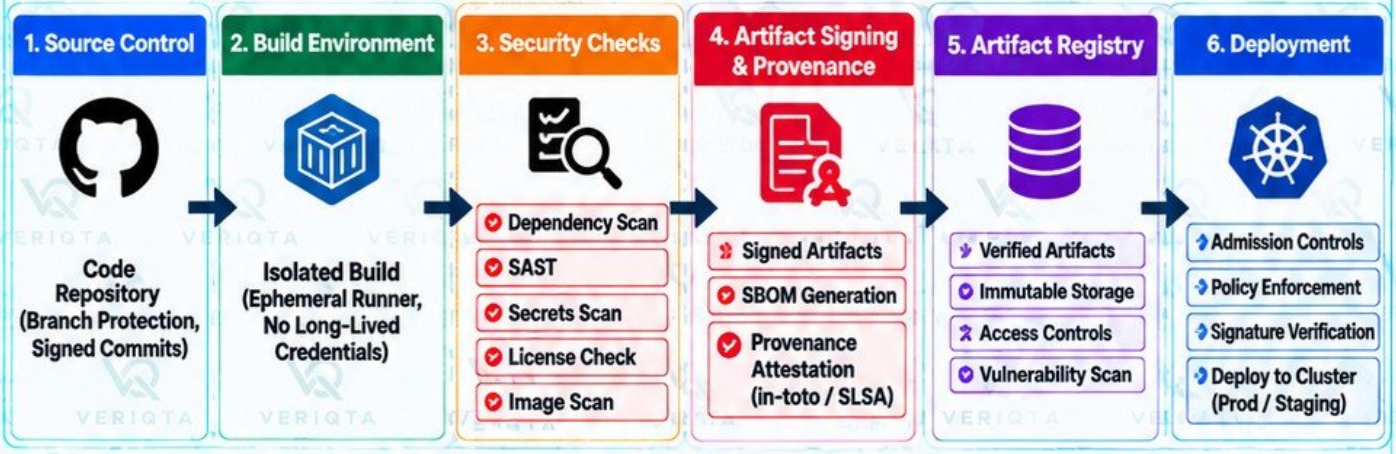
Project Goal

Create isolated build environments, short-lived credentials, signed artifacts, provenance attestations, dependency verification, and deployment admission controls.

Engineering Challenge

Block artifact substitution and compromised build identities.

Zero-Trust CI/CD Supply Chain Architecture



Key Security Components

- Short-Lived Credentials (OIDC / STS)
- Ephemeral Build Runners
- Artifact Signing (Cosign)
- SBOM (Software Bill of Materials)
- Provenance (SLSA / in-toto)
- Dependency Verification (Snyk / Grype)
- Policy as Code (OPA / Conftest)
- Deployment Admission (Kyverno / Gatekeeper)

Implementation Areas

- Secure source control and branch protection
- Isolated and ephemeral build environments
- Short-lived cloud credentials (OIDC / STS)
- Automated security and dependency scanning
- Artifact signing and provenance generation
- SBOM creation and storage
- Policy-as-code for deployment controls
- Admission policies in Kubernetes
- Immutable artifact registry
- Audit logging and traceability

Success Criteria

- [x] No long-lived credentials in CI/CD
- [x] All artifacts signed and verified
- [x] SBOM and provenance available
- [x] Blocked unsigned or modified artifacts
- [x] Policy enforcement in deployment
- [x] Dependency vulnerabilities detected
- [x] Reproducible and auditable builds
- [x] End-to-end traceability from commit to production

Failure Scenarios to Test

- [x] Use a tampered artifact
- [x] Deploy an unsigned image
- [x] Inject a vulnerable dependency
- [x] Compromise the build identity
- [x] Bypass branch protection
- [x] Modify artifact after signing
- [x] Attempt deployment without required attestations
- [x] Use outdated or revoked credentials
- [x] Policy violation in deployment

06 Progressive Delivery and Automated Rollback Platform

Project Goal

Implement canary analysis, blue-green deployment, traffic shifting, deployment metrics, error-budget gates, and automated rollback.

Engineering Challenge

Detect a release that passes health checks but increases customer-facing errors.

Progressive Delivery Pipeline

Code Commit → Build & Test (CI) → Security Scan → Build Artifact → Artifact Registry → Deploy (Progressive) → Monitor & Analyze → Auto Rollback

Progressive Delivery Strategies

Canary Analysis
- Small % of traffic
- Compare metrics
- Gradual rollout

Blue-Green Deployment
- Parallel environments
- Instant switch
- Quick rollback

Traffic Shifting
- Weighted routing
- Real-time adjustment
- Auto revert

Deployment Metrics
- Latency
- Error rate
- Throughput

Error-Budget Gates
- SLO tracking
- Guardrails
- Block if over budget

Automated Rollback
- Detect anomalies
- Revert automatically
- Notify team

Key Components

- Progressive delivery controller (e.g., Argo Rollouts, Flagger, Spinnaker)
- Containerized applications (Docker)
- Kubernetes cluster (or EKS/GKE/AKS)
- Monitoring and observability (Prometheus, Grafana)
- Service mesh / ingress (Istio, NGINX)
- Policy and security controls
- Automated rollback triggers and runbooks

Tools and Technologies

Kubernetes · Argo Rollouts · Prometheus · Grafana · Istio · NGINX · Docker · GitHub

Success Criteria

- Successful progressive rollouts (canary / blue-green)
- Accurate traffic shifting and metric validation
- Error-budget gates working as expected
- Automatic rollback when thresholds are breached
- Zero or minimal customer impact
- Clear deployment and rollback logs

Failure Scenarios to Test

- Increased error rate (passes health checks)
- Latency spike during rollout
- Partial service failure
- Misconfigured traffic routing
- Rollback loop
- Monitoring or alerting failure

07 GitOps Fleet Management Across Kubernetes Clusters

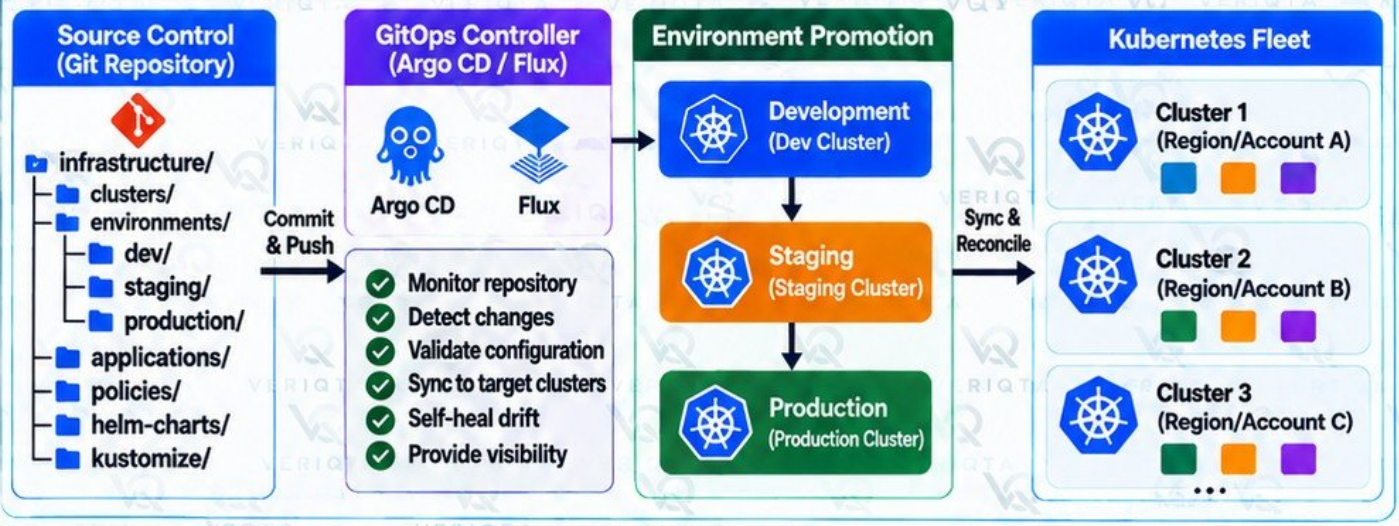
Project Goal

Build a GitOps control architecture with environment promotion, reconciliation, configuration drift detection, policy validation, and cluster-level failure isolation.

Engineering Challenge

Recover from bad configuration propagation without creating a fleet-wide outage.

GitOps Fleet Architecture



Key Components

- Git repository structure (environments, clusters, apps)
- GitOps controller (Argo CD or Flux)
- Environment promotion workflow
- Declarative configuration (Helm / Kustomize)
- Policy validation (OPA / Kyverno)
- Configuration drift detection and alerting
- Cluster-level isolation and failure containment
- Observability and audit logging

Tools and Technologies

Argo CD · Flux · Kubernetes · Helm · Kustomize · OPA · Kyverno · Prometheus

Success Criteria

- [x] Reliable environment promotion
- [x] Automatic reconciliation and self-healing
- [x] Configuration drift detected and alerted
- [x] Policies enforced before deployment
- [x] Cluster isolation works as expected
- [x] Clear audit trail of all changes
- [x] Rollback to previous version when needed
- [x] No fleet-wide impact from bad configurations

Failure Scenarios to Test

- [x] Push a bad configuration to Git
- [x] Invalid manifest that breaks workloads
- [x] Policy violation attempt (e.g., privileged container)
- [x] Accidental deletion of a critical resource
- [x] Configuration drift in a cluster
- [x] Application version mismatch across clusters
- [x] Controller outage (Argo CD / Flux)
- [x] Recover without affecting other clusters

08 Internal Developer Platform with Self-Service Provisioning

Project Goal

Build a developer portal and platform API for standardized service creation, infrastructure provisioning, access governance, ownership, and golden paths.

Engineering Challenge

Provide developer autonomy without uncontrolled infrastructure access.

Platform Architecture

Developers

- Request service
- Use golden paths
- Self-service provisioning
- View ownership
- Track approvals

Developer Portal

- Service templates
- Catalog
- Golden paths
- Documentation
- Self-service UI
- Status tracking

Platform API

- Service creation
- Infrastructure provisioning
- Access governance
- Policy enforcement
- Audit logging
- Integration with tools

Provisioning & Automation

- Infrastructure as Code
- Standardized modules
- Automated deployment
- Environment setup
- Secrets management
- CI/CD integration

Cloud & Platform Resources

- Compute
- Networking
- Databases
- Security controls
- Monitoring & logging
- Cost controls

Key Components

- Developer portal (e.g., Backstage)
- Service templates and golden paths
- Platform API and catalog
- Infrastructure provisioning (Terraform)
- Access governance (RBAC / IAM)
- Policy enforcement (OPA / Sentinel)
- Secrets management (Vault)
- CI/CD integration
- Audit logging and ownership tracking
- Monitoring and cost controls

Example Service Provisioning Flow

1 Select Template

- Choose service type (e.g., web app, API)
- Predefined templates
- Golden path guidance

2 Configure Parameters

- Environment (dev/staging/prod)
- Team and ownership
- Resource requirements
- Automatic validation

3 Automated Provisioning

- Infrastructure as Code
- Security and policy checks
- Environment deployment
- Access configuration
- Notifications

4 Service Ready

- Application deployed
- Access granted
- Monitoring enabled
- Ownership recorded
- Ongoing lifecycle management

Success Criteria

- Developers can provision services independently
- Standardized and secure infrastructure
- No uncontrolled infrastructure access
- Auditable access and ownership
- Policy enforcement works as expected
- Services follow golden paths
- Reduced manual platform engineering workload
- Faster and consistent service delivery

Failure Scenarios to Test

- Attempt to provision resources outside allowed templates
- Privilege escalation attempt through the portal
- Bypass policy enforcement
- Deploy non-compliant infrastructure
- Use excessive resource quotas
- Access another team's resources
- Missing ownership or audit records
- Misconfigured template leading to security risk
- Ensure platform prevents and alerts on all violations

09 SLO-Driven Reliability Engineering Platform

Project Goal

Build service-level indicators, burn-rate alerts, error-budget policies, release controls, and reliability reporting.

Engineering Challenge

Connect deployment decisions to actual user-facing reliability.

SLO-Driven Reliability Architecture

User Traffic

Web / Mobile
API / Services

Observability Stack

Prometheus (Metrics)
Grafana (Dashboards)
Loki (Logs)

- Collect metrics
- Monitor service health
- Track SLIs in real time

SLO & Error Budget

- Define SLIs
- Set SLOs
- Calculate error budget
- Track burn rate
- Expose reliability status

Alerts & Policies

- Burn-rate alerts
- Error-budget policies
- Automated notifications
- Release controls
- Escalation rules

Deployment Decisions

- Allow-deployment
- Block-deployment
- Automatic rollback
- Require approval
- Based on SLO status

Key Components

- Service-level indicators (SLIs)
- Service-level objectives (SLOs)
- Error-budget tracking
- Burn-rate alerting (multi-window)
- Release policies and gates
- Reliability dashboards and reporting
- Integration with CI/CD
- Incident response and post-incident reviews
- User-facing reliability metrics

Example SLO Metrics

Latency SLI
- p50, p95, p99
- Response time

Availability SLI
- Successful requests
- Uptime percentage

Error Rate SLI
- 4xx / 5xx rate
- Application errors

Saturation SLI
- CPU, memory, disk
- Queue length

Reliability Workflow

1. Collect and analyze SLIs
2. Track error budget consumption
3. Trigger burn-rate alerts
4. Evaluate deployment against reliability policies
5. Allow, block, or rollback release
6. Monitor impact on user experience
7. Update reliability reports

Success Criteria

- Accurate and actionable SLOs
- Reliable burn-rate alerting
- Deployment decisions enforce reliability policies
- Clear visibility into error budget status
- Reduced customer-facing incidents
- Evidence of improved reliability over time
- Dashboards and reports for stakeholders
- Integration with CI/CD and automated controls

Failure Scenarios to Test

- Increase error rate without breaking health checks
- Latency spike during and after deployment
- Rapid error budget burn
- Deployment during an active incident
- False positives in alerting
- Release that degrades user experience
- Rollback triggered by reliability policy
- Sustained partial outage

10 Distributed Observability and Root-Cause Investigation

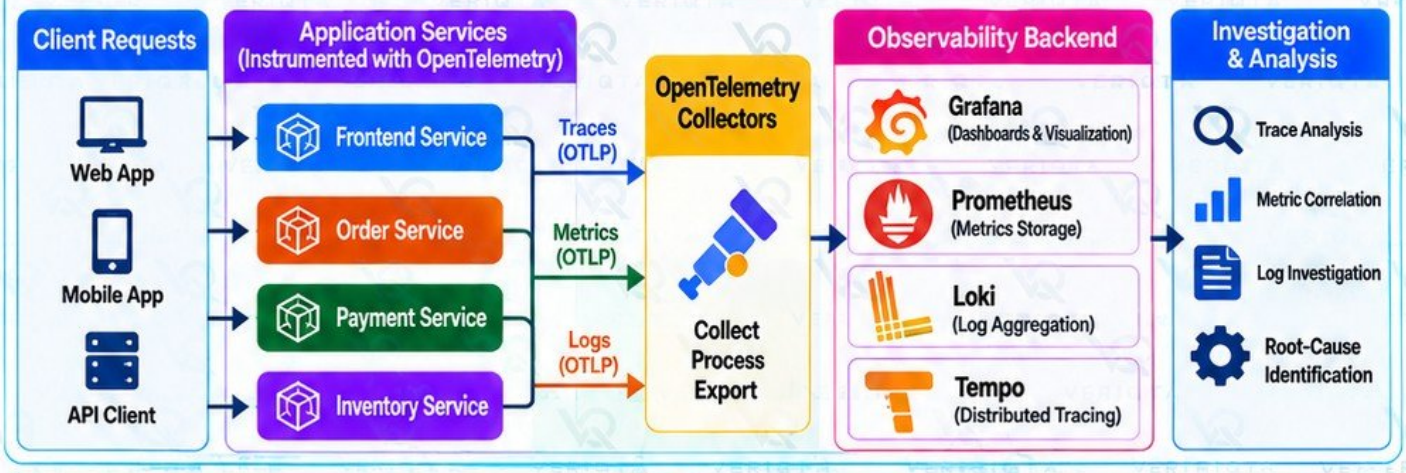
Project Goal

Instrument a distributed application with OpenTelemetry, correlate traces, metrics, and logs, and investigate latency across service dependencies.

Engineering Challenge

Identify the true bottleneck during partial degradation without relying on dashboards alone.

Distributed Observability Architecture



Key Components

- OpenTelemetry instrumentation (traces, metrics, logs)
- Distributed tracing (e.g., Jaeger / Tempo)
- Metrics collection and storage (Prometheus)
- Log aggregation (Loki)
- Service dependency mapping
- Correlation and root-cause analysis
- Alerting and anomaly detection
- Dashboards and SLO monitoring

Tools and Technologies

OpenTelemetry | Prometheus | Grafana | Loki

Tempo | Jaeger | Kubernetes | Docker

Success Criteria

- End-to-end traces across all services
- Accurate service dependency map
- Ability to identify the true bottleneck
- Correlated logs, metrics, and traces
- Meaningful dashboards and alerts
- Investigation completed using real data
- Root cause validated (not just symptoms)
- Clear documentation of findings and actions

Failure Scenarios to Test

- Latency increase in a downstream dependency
- Partial service degradation (some requests fail)
- Misleading metrics or logs
- High error rate without clear source
- Database or external API slowdown
- Resource exhaustion in a single service
- Network latency between services
- False correlation leading to incorrect root cause

11 Chaos Engineering and Automated Resilience Validation

Project Goal

Create controlled failure experiments for node loss, DNS disruption, network latency, dependency timeouts, and availability-zone failure.

Engineering Challenge

Demonstrate that recovery mechanisms work under measurable failure conditions.

Chaos Engineering Architecture

Failure Enperiments

- **Node Loss** — Terminate nodes or instances
- **DNS Disruption** — Introduce DNS failures or latency
- **Network Latency** — Add latency, packet loss or network partition
- **Dependency Timeouts** — Slow or fail dependencies
- **Availability-Zone Failure** — Simulate AZ outage

Chaos Tooling

- Chaos Mesh
- Litmus
- Gremlin
- AWS FIS
- Custom Experiments

Target Environment

- Kubernetes Cluster
- Application Services
- Databases & Dependencies
- Multi-Region / Multi-AZ

Observability & Validation

- Prometheus (Metrics)
- Grafana (Visualization)
- Loki (Logs)
- Jaeger (Tracing)

Resilience Outcomes

- Automatic recovery
- Failover works as expected
- Service remains within SLO
- Alerts triggered correctly
- Recovery time meets targets
- No data loss (based on RPO)

Key Components

- Controlled failure experiments
- Kubernetes chaos tools (Chaos Mesh / Litmus)
- AWS Fault Injection Simulator (FIS)
- Network and DNS disruption testing
- Service dependency failure scenarios
- Observability (metrics, logs, traces)
- SLO validation and reporting
- Automated experiment scheduling
- Safety controls and blast radius limits
- Result analysis and improvements

Example Failure Experiment Flow

1 Define Experiment
- Select failure type
- Define scope
- Set blast radius
- Define success criteria (SLO)

2 Run Experiment
- Inject failure
- Monitor impact
- Track metrics
- Observe system behavior

3 Validate Recovery
- Check failover
- Verify recovery time (RTO)
- Verify data loss (RPO)
- Confirm SLOs

4 Report Results
- Generate report
- Analyze findings
- Identify gaps
- Update runbooks

5 Automate & Improve
- Schedule tests
- Add new scenarios
- Tune alerts
- Implement improvements

Success Criteria

- [ ] All planned failure scenarios executed
- [ ] Services recover automatically
- [ ] Recovery time meets defined RTO
- [ ] No data loss beyond defined RPO
- [ ] Alerts trigger and provide useful signals
- [ ] System remains within SLO during failures
- [ ] Clear evidence and reports of recovery
- [ ] Actionable improvements implemented

Failure Scenarios to Test

- [ ] Kill Kubernetes nodes or cloud instances
- [ ] Simulate DNS failure or high DNS latency
- [ ] Introduce network latency and packet loss
- [ ] Force dependency timeouts (database / external API)
- [ ] Simulate availability-zone outage
- [ ] Combine multiple failures (e.g., AZ + network latency)
- [ ] Test during peak traffic conditions
- [ ] Verify recovery under repeated failures

12 Production Incident Response and Automated Remediation

Project Goal

Build an incident detection, classification, escalation, runbook execution, and recovery verification system with human approval for high-risk actions.

Engineering Challenge

Prevent automated remediation from amplifying the original incident.

Incident Response Architecture

Detection & Alerting

- Metrics (Prometheus)
- Logs (Loki)
- Traces (Tempo)
- Anomaly detection
- Alert rules and thresholds

Classification

- Auto-triage
- Severity levels (P1–P4)
- Service and impact detection
- Enrichment with context
- Deduplication and noise reduction

Escalation & Notification

- On-call routing
- Slack / Teams alerts
- PagerDuty integration
- Escalation policies
- Stakeholder notifications

Runbook Execution

- Automated runbooks
- Safe remediation actions
- Human approval for high-risk changes
- Change tracking
- Rollback capability

Recovery Verification

- Validate service health
- Check SLOs and error rates
- Confirm downstream dependencies
- Close incident automatically
- Post-incident reporting

Key Components

- Monitoring and alerting (Prometheus, Alertmanager)
- Incident management (PagerDuty / Opsgenie)
- ChatOps (Slack / Teams)
- Automated runbooks (Ansible / Kubernetes / Scripts)
- Approval workflows for high-risk actions
- Recovery verification and health checks
- Incident timeline and audit logs
- Post-incident analysis and reporting
- Integration with on-call schedules and rotation
- Safety controls to prevent cascading failures

Example Incident Response Flow

1 Alert Triggered
- High error rate detected
- Alert sent to on-call
- Incident created automatically

2 Triage & Classify
- Auto-classify severity
- Identify affected services
- Enrich with logs, metrics, traces

3 Execute Runbook
- Run safe remediation
- Request approval for high-risk actions
- Track execution steps

4 Verify Recovery
- Check service health
- Validate SLOs
- Confirm no further impact

5 Close & Learn
- Close incident
- Generate report
- Identify improvements
- Update runbooks

Success Criteria

- [x] Incidents detected and classified accurately
- [x] Correct escalation to on-call engineers
- [x] Automated remediation works for safe actions
- [x] Human approval required for high-risk changes
- [x] Recovery verified before closure
- [x] No amplification of the original incident
- [x] Clear incident timeline and audit trail
- [x] Post-incident reports with actionable improvements
- [x] Reduced MTTR (Mean Time to Recovery)

Failure Scenarios to Test

- [x] Incorrect alert classification (false positive/negative)
- [x] Runbook makes the problem worse
- [x] Automated remediation without approval
- [x] Cascading failures across dependent services
- [x] Repeated remediation loops
- [x] Missing rollback after failed remediation
- [x] Service recovers but downstream dependency still failing
- [x] On-call escalation not triggered
- [x] Incomplete incident closure
- [x] Loss of data or configuration during remediation

13 Multi-Cluster Service Networking and Traffic Resilience

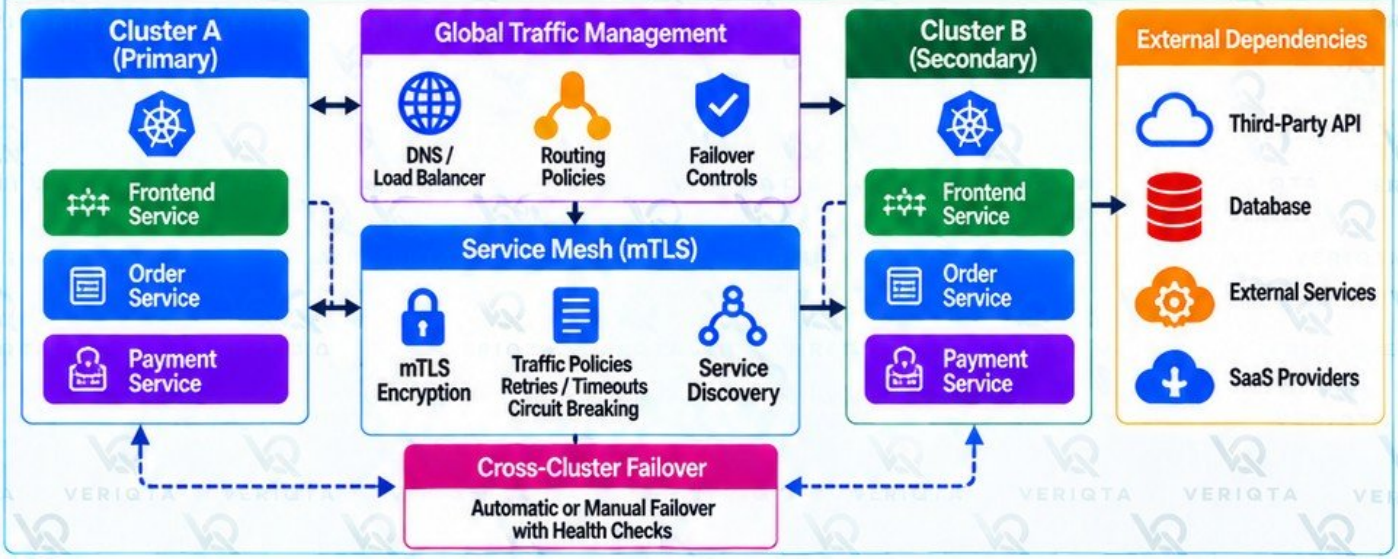
Project Goal

Engineer service-to-service connectivity, mTLS, routing policies, bounded retries, circuit breaking, and cross-cluster failover.

Engineering Challenge

Prevent retry storms and connection exhaustion during dependency degradation.

Multi-Cluster Service Networking Architecture



Key Components

- Multi-cluster networking (e.g., Cilium / Istio)
- Service-to-service mTLS
- Global traffic management (DNS / LB)
- Routing policies and traffic splitting
- Bounded retries and timeouts
- Circuit breaking and bulkheads
- Cross-cluster failover and health checks
- Service discovery and secure communication
- Observability and traffic monitoring
- Connection and resource limits

Traffic Resilience Flow

1 Service Request	2 Traffic Policies	3 Service Processing	4 Failure Handling	5 Resilient Outcome
Client sends request	Apply routing policies	Service handling request	Detect unhealthy service	Request succeeds in healthy cluster
DNS / LB routes to healthy cluster	Set timeouts and retries	Monitor health and latency	Trigger failover or reroute	No connection exhaustion
mTLS secured connection	Use circuit breaking	Apply resource limits	Prevent retry storms	System remains stable

Success Criteria

- Secure service-to-service communication (mTLS)
- Reliable cross-cluster connectivity
- Correct routing during partial or full failure
- Bounded retries to prevent overload
- Circuit breaking works as expected
- Automatic or manual failover tested successfully
- No retry storms or connection exhaustion
- Services remain available and performant
- Clear monitoring and alerting
- Documented test results and recovery time

Failure Scenarios to Test

- Dependency service latency or timeouts
- High error rate from external API
- Simulate complete cluster failure
- Retry storm due to misconfigured clients
- Connection exhaustion (too many open connections)
- Circuit breaker not triggering
- DNS or load balancer misrouting traffic
- Partial service degradation (some instances failing)
- Cross-cluster failover failure
- Network partition between clusters

14 Enterprise Secrets and Workload Identity Platform

Project Goal

Implement workload federation, dynamic credentials, certificate automation, rotation, access auditing, and revocation.

Engineering Challenge

Contain credential theft and recover from compromised workload identities.

Workload Identity and Secrets Architecture

Identity Sources

- **Human Users** — SSO / IAM
- **Workloads** — Kubernetes / CI/CD / Services
- **External Providers** — Cloud / SaaS / Third-Party

Federation & Identity Management

- **Workload Federation** — OIDC / SAML
- **Identity Broker** — Exchange tokens for short-lived credentials
- **Dynamic Credentials** — Time-bound and scoped access

Secrets Management

- **Secrets Store** — HashiCorp Vault / AWS Secrets Manager / Azure Key Vault
- **Certificate Automation** — Issue, rotate, renew certificates
- **Rotation & Revocation** — Automated rotation and instant revocation

Workload Access

- **Applications and Services** — Retrieve secrets at runtime
- **Kubernetes Workloads** — Workload identity (e.g., IRSA / Workload Identity Federation)
- **CI/CD Pipelines** — Short-lived tokens for builds and deployments

Security & Audit

- **Access Auditing** — Log all secret access and identity usage
- **Policy Enforcement** — RBAC / ABAC / Least privilege
- **Monitoring & Alerts** — Detect anomalies and unauthorized access

Key Components

- Workload identity federation (OIDC / SAML)
- Dynamic, short-lived credentials
- Secrets management (Vault / AWS / Azure)
- Certificate automation and rotation
- Access control and least privilege
- Audit logging and monitoring
- Secret revocation and incident response
- Integration with Kubernetes and CI/CD
- Detection of anomalous secret usage
- Automated compliance reporting

Example Workflow

1 — Workload Authenticates
- Use workload identity (OIDC)
- Exchange for short-lived token

2 — Retrieve Secrets
- Access secrets from Vault
- Apply least privilege policies

3 — Use in Application
- Inject secrets at runtime
- No hardcoded credentials

4 — Rotate & Monitor
- Automatic rotation
- Monitor usage and anomalies

5 — Revoke if Compromised
- Detect suspicious activity
- Revoke credentials and invalidate tokens

Success Criteria

- [ ] All workloads use federated identity (no static keys)
- [ ] Secrets automatically rotated on schedule
- [ ] Compromised credentials can be revoked immediately
- [ ] Access is audited and logged
- [ ] Least privilege enforced for all services
- [ ] Successful certificate automation and renewal
- [ ] No hardcoded secrets in code or pipelines
- [ ] Alerts trigger on anomalous secret access
- [ ] Incident recovery completed with minimal impact
- [ ] Compliance and audit reports generated successfully

Failure Scenarios to Test

- [ ] Stolen workload identity token used from unauthorized IP
- [ ] Excessive secret access (unusual pattern)
- [ ] Expired or revoked token still being accepted
- [ ] Certificate renewal failure
- [ ] Misconfigured access policy allowing broad permissions
- [ ] Secrets exposed in logs or environment variables
- [ ] Simulated insider threat with unauthorized access
- [ ] Revocation of compromised credentials and verification of access denial
- [ ] Recovery of services after credential compromise
- [ ] Audit alerts and notifications triggered correctly

15 High-Availability Database and Stateful Workload Recovery

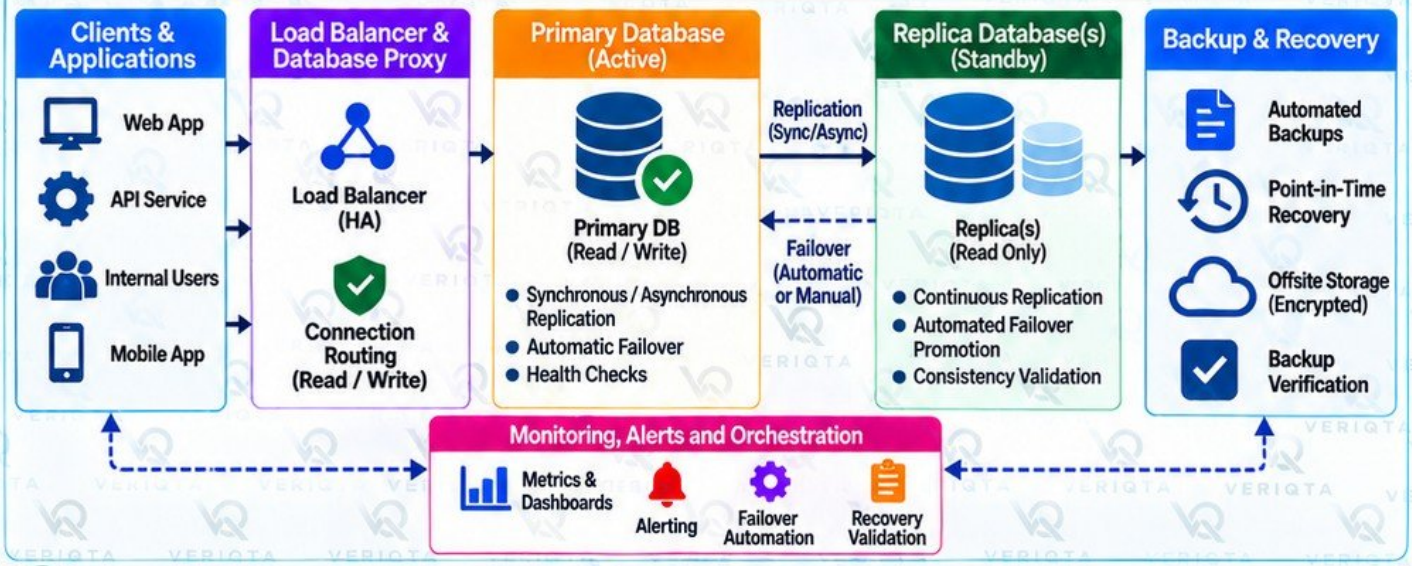
Project Goal

Deploy a resilient database-backed system with replication, failover orchestration, backup verification, point-in-time recovery, and consistency checks.

Engineering Challenge

Prove recovery correctness and protect against split-brain and data loss.

High-Availability Database Architecture



Key Components

- Database replication (sync / async)
- High-availability architecture (multi-AZ / multi-region)
- Automated failover and health checks
- Backup strategy and verification
- Point-in-time recovery (PITR)
- Consistency checks and data validation
- Split-brain prevention (quorum / fencing)
- Monitoring, alerting, and incident response
- Stateful workload orchestration (Kubernetes / VMs)
- Disaster recovery testing and documentation

Recovery Workflow

1 Detect Failure
- Monitor health and replication
- Alert on failure
- Identify impact and scope

2 Failover Orchestration
- Trigger automatic or manual failover
- Promote replica
- Update routing and DNS

3 Validate Recovery
- Check service health
- Validate data consistency
- Run application tests

4 Restore if Needed
- Use point-in-time recovery
- Restore from verified backup
- Verify data integrity

5 Moniter & Stabilize
- Monitor for anomalies
- Confirm no split-brain
- Document and close incident

Success Criteria

- [x] Automatic failover works as expected
- [x] Data remains consistent after failover
- [x] Backups are successfully verified and restorable
- [x] Point-in-time recovery restores correct data
- [x] No split-brain during failures
- [x] Application recovers with minimal downtime
- [x] Recovery process is documented and repeatable
- [x] Monitoring and alerts trigger correctly
- [x] Regular disaster recovery tests are successful
- [x] Data loss is prevented

Failure Scenarios to Test

- [x] Primary database node failure
- [x] Network partition causing split-brain risk
- [x] Replica promotion failure
- [x] Corrupted or incomplete backups
- [x] Point-in-time recovery to specific timestamp
- [x] Data inconsistency after failover
- [x] Storage failure or disk corruption
- [x] Cross-region failover (multi-AZ / multi-region)
- [x] High write load during failover
- [x] Human error in failover or recovery process

16 Kubernetes Capacity Engineering and Autoscaling at Scale

Project Goal

Combine workload profiling, HPA, node provisioning, topology constraints, capacity forecasting, and cost analysis.

Engineering Challenge

Handle demand spikes without cascading scheduling failures or uncontrolled spending.

Capacity Engineering Architecture

Workload Profiling & Metrics

- Application Metrics (CPU / Memory / Custom)
- Workload Profiling (Requests / Limits)
- Historical Usage (Prometheus)
- Traffic Patterns (Coming Spikes)

Autoscaling & Node Provisioning

- Horizontal Pod Autoscaler (HPA)
- Vertical Pod Autoscaler (VPA)
- Cluster Autoscaler (CA)
- Karpenter (Node Provisioning)

Scheduling & Topology Constraints

- Node Selectors & Affinity
- Pod Anti-Affinity (High Availability)
- Topology Spread Constraints (Multi-AZ / Multi-Region)
- Taints & Tolerations (Workload Isolation)

Capacity Planning & Cost Analysis

- Capacity Forecasting (Trend Analysis)
- Resource Optimization
- Cost Analysis (CPU / Memory / Nodes / Cloud)
- Idle Resource Detection

Outcomes

- Automatic Scaling
- High Availability
- Optimal Resource Usage
- Controlled Cloud Costs
- No Scheduling Failures
- Stable Performance during Spikes

Key Components

- Workload profiling and metrics collection
- Horizontal Pod Autoscaler (HPA)
- Vertical Pod Autoscaler (VPA)
- Cluster Autoscaler / Karpenter
- Node provisioning and instance types
- Topology constraints (affinity, anti-affinity, spread)
- Resource requests and limits
- Capacity forecasting and trend analysis
- Cost optimization and budget controls
- Monitoring, alerting and reporting
- Load and spike testing
- Failure scenarios and recovery validation

Example Capacity Scaling Flow

1 Detect Demand Spike
- Monitor metrics
- Detect traffic increase
- Identify affected services

2 Scale Workloads
- HPA scales pods
- VPA optimizes resources
- Maintain performance targets

3 Provision Nodes
- Cluster Autoscaler or Karpenter adds nodes
- Select optimal instance types
- Apply topology constraints

4 Maintain Stability
- Balance workloads
- Prevent scheduling failures
- Ensure high availability
- Control costs

5 Validate Outcomes
- Performance meets targets
- No pod evictions
- Costs within budget
- System remains stable

Success Criteria

- [ ] Workloads automatically scale during demand spikes
- [ ] No scheduling failures or pending pods
- [ ] Resources are right-sized and cost-efficient
- [ ] High availability maintained across zones/regions
- [ ] Capacity forecasting is accurate
- [ ] Controlled cloud spending (no uncontrolled costs)
- [ ] Performance remains stable under high load
- [ ] Recovery from scale-down and scale-up events works
- [ ] Monitoring and alerts function correctly
- [ ] Documentation with test results and cost analysis

Failure Scenarios to Test

- [ ] Rapid traffic spike (e.g., 5–10x normal load)
- [ ] Node exhaustion and insufficient capacity
- [ ] Incorrect resource requests and limits
- [ ] Autoscaler failure or misconfiguration
- [ ] Pods stuck in Pending state
- [ ] Topology constraint conflict (unschedulable)
- [ ] Cost overrun due to uncontrolled scaling
- [ ] Instance type capacity shortage (spot interruptions)
- [ ] Scale-down causing application instability
- [ ] Regional/AZ failure during high load

17 Cloud FinOps and Reliability-Aware Cost Engineering

Project Goal

Build cost attribution, workload rightsizing, anomaly detection, capacity commitments, and cost-versus-availability analysis.

Engineering Challenge

Reduce infrastructure waste without weakening resilience or performance.

FinOps and Cost Engineering Architecture

Data Collection and Visibility

- Cloud Provider Billing Data (AWS / Azure / GCP)
- Resource Usage (Metrics & Logs)
- Tags and Cost Allocation
- Multi-Account / Multi-Region Data

Cost Attribution and Analysis

- Cost Allocation by Team / Service / Environment
- Cost Breakdown (Compute / Storage / Network / Others)
- Tag Governance and Standards
- Chargeback / Showback Reporting

Optimization and Rightsizing

- Workload Rightsizing (CPU / Memory / Instances)
- Idle Resource Detection
- Modernization Opportunities (Managed Services / Serverless)
- Automated Scaling Policies

Anomaly Detection and Forecasting

- Cost Anomaly Detection (AI / ML / Rules)
- Usage Forecasting and Capacity Planning
- Cost vs Availability Analysis
- Budget Alerts and Thresholds

Reliability-Aware Decision Making

- Balance Cost with Resilience
- Evaluate Trade-offs (Availability vs Cost)
- Capacity Commitments (RI / Savings Plans)
- Optimize Without Impacting Performance

Key Components

- Cloud cost visibility and attribution
- Tagging strategy and governance
- Workload rightsizing and optimization
- Anomaly detection and alerting
- Capacity forecasting and demand planning
- Reserved instances and savings plans
- Cost-versus-availability analysis
- Automated scaling and scheduling policies
- Multi-account and multi-region cost management
- Reporting, dashboards, and stakeholder communication

Example Cost Optimization Flow

1 Identify Current Spend
- Collect and analyze usage data
- Break down costs by service and team
- Find top cost drivers

2 Detect Optimization Opportunities
- Identify idle and underutilized resources
- Find right-sizing candidates
- Detect anomalies and unusual spikes

3 Apply Optimizations
- Right-size workloads
- Use appropriate instance types
- Implement auto-scaling and schedules
- Leverage reserved capacity

4 Validate Impact
- Ensure no performance loss
- Confirm availability and SLOs remain met
- Track cost reduction
- Monitor over time

5 Continuous Improvement
- Refine policies
- Update forecasts
- Adjust capacity commitments
- Improve cost allocation and governance

Success Criteria

- Accurate cost attribution by team and service
- Identify and eliminate unnecessary spend
- Workloads right-sized without performance impact
- Anomaly detection catches unusual cost spikes
- Capacity commitments optimized for usage patterns
- Availability and reliability targets maintained
- Measured and verified cost reduction
- Clear reporting and stakeholder visibility
- Automated policies reduce ongoing waste
- Sustainable and repeatable cost optimization process

Failure Scenarios to Test

- Unexpected cost spike (e.g., runaway resource creation)
- Idle resources running for long periods
- Over-provisioned instances or storage
- Under-provisioning leading to performance and availability issues
- Wrong tagging or missing tags causing incorrect attribution
- Anomaly detection false positives or missed detections
- Reserved instance mismatch with actual usage
- Auto-scaling misconfiguration causing high costs
- Cost optimization reducing redundancy and breaking SLOs
- Forecasting errors leading to capacity shortages or waste

18 Continuous Compliance and Infrastructure Governance

Project Goal

Engineer automated evidence collection, cloud configuration assessment, identity reviews, control mapping, and exception workflows.

Engineering Challenge

Detect control failures and prove remediation using verifiable evidence.

Compliance and Governance Architecture

Data Sources and Connectors

- Cloud Providers (AWS / Azure / GCP)
- Infrastructure (Kubernetes / VMs)
- Configuration Management (Terraform / Ansible)
- Identity & Access (IAM / SSO / AD)
- Logs & Audit Data (CloudTrail / Audit Logs)

Assessment and Scanning

- Configuration Assessment (CIS / Best Practices)
- Vulnerability Scanning (Images / Instances)
- Policy Compliance (IaC / Cloud Resources)
- Continuous Monitoring (Real-time & Scheduled)

Control Mapping and Evaluation

- Control Mapping (SOX / ISO 27001 / NIST / CIS)
- Compliance Evaluation (Pass / Fail / Risk)
- Risk Scoring and Prioritization
- Drift Detection (Against Approved Baseline)

Remediation and Exception Management

- Automated Remediation (Fix or Create Ticket)
- Exception Workflows (Approval Process)
- Change Tracking and Audit Trail
- Evidence Collection (Screenshots / Logs / Reports)

Reporting and Governance

- Compliance Dashboards (Real-time)
- Audit Reports and Evidence (Automated)
- Stakeholder Access (Security / Audit / Leadership)
- Continuous Improvement (Policies / Controls / Process Updates)

Key Components

- Cloud configuration assessment and policy enforcement
- Identity and access reviews (users, roles, permissions)
- Control mapping (SOX / ISO 27001 / NIST / CIS)
- Automated evidence collection and storage
- Continuous compliance monitoring and alerting
- Exception management and approval workflows
- Drift detection and remediation validation
- Audit-ready reporting and dashboards
- Integration with ticketing and change management
- Governance, risk and compliance (GRC) processes

Example Compliance Workflow

1 Scan Resources
- Discover cloud resources
- Run configuration and security scans
- Collect identity and access data

2 Evaluate Controls
- Check against policies/standards
- Identify misconfigurations
- Assign risk levels

3 Remediate or Escalate
- Auto-remediate where possible
- Create tickets for manual fixes
- Route exceptions for approval

4 Collect Evidence
- Capture logs, screenshots
- Generate reports
- Store in audit repository

5 Monitor Continuously
- Track compliance status
- Alert on new violations
- Verify remediation and close findings

Success Criteria

- Automated collection of accurate and complete evidence
- High compliance score across critical controls
- Misconfigurations detected and remediated
- Identity and access reviews completed regularly
- Exceptions tracked and approved with full audit trail
- Audit reports generated automatically
- No critical control failures during testing
- Remediation verified with documented evidence
- Stakeholders can access real-time compliance status
- Sustainable continuous compliance process in place

Failure Scenarios to Test

- Misconfigured cloud resources (public access, open ports)
- Excessive permissions or orphaned accounts
- Disabled or missing logging and audit trails
- Policy violations in infrastructure as code
- Unauthorized changes to critical resources
- Failed remediation attempts
- Exception requests without proper approval
- Evidence collection failures (missing or incomplete logs)
- Audit report generation errors
- Repeated compliances drift after remediation

19 Production Migration with Zero-Downtime Cutover

Project Goal

Migrate a stateful production service using parallel environments, compatibility controls, traffic shifting, data validation, rollback, and reconciliation.

Engineering Challenge

Avoid data divergence and prevent irreversible cutover decisions.

Zero-Downtime Migration Architecture

1. Parallel Environments

- Source (Environment A)
- Destination (Environment B)
- Identical architecture
- Data replication (continuous sync)
- Compatibility checks
- Feature parity validation

2. Data Synchronization

- Continuous replication
- Change data capture (CDC)
- Data consistency checks
- Validate schema and data integrity
- Monitor replication lag

3. Traffic Shifting

Load Balancer

Old / New

- Gradual traffic shift (canary / percentage)
- Observe performance and errors
- Automated rollback triggers
- Maintain session persistence (if required)

4. Validation and Cutover

- Data validation (record counts, checksums)
- Application health and functionality tests
- Compare logs and metrics
- Business validation
- Final cutover with zero downtime

5. Rollback and Reconciliation

- Instant rollback plan
- Data reconciliation (if needed)
- Verify no data loss or corruption
- Re-route traffic to original environment
- Document and close migration

Key Components

- Parallel environments (source and destination)
- Continuous data replication (CDC)
- Compatibility and schema validation
- Traffic shifting (load balancer / service mesh)
- Application health monitoring
- Data validation and reconciliation
- Automated rollback and feature flags
- Observability (logs, metrics, traces)
- Stakeholder communication and approvals
- Post-migration monitoring and optimization

Example Migration Flow

1 Prepare Environments
- Build and test destination
- Ensure feature parity
- Set up replication

2 Sync Data
- Enable CDC
- Monitor lag
- Validate data integrity

3 Shift Traffic
- Start with small percentage
- Monitor metrics and errors
- Increase gradually

4 Validate
- Run automated tests
- Compare data
- Get business sign-off

5 Complete Cutover
- Route 100% traffic
- Continue monitoring
- Keep rollback ready

Success Criteria

- Zero downtime during migration
- No data loss or corruption
- Source and destination data remain consistent
- Application performance is maintained or improved
- Successful traffic shift and cutover
- Rollback plan tested and ready
- All validation checks pass
- Business users confirm successful migration
- Post-migration stability (no critical issues)
- Documentation and lessons learned completed

Failure Scenarios to Test

- Data divergence between source and destination
- Replication lag causing stale data
- Application errors during traffic shift
- Compatibility issues (schema / version mismatch)
- Performance degradation after cutover
- Partial traffic cutover failures
- Rollback failure or data inconsistency after rollback
- Unintended writes to old environment
- Irreversible cutover without full validation
- Monitoring gaps hiding migration issues

20 Enterprise Platform Failure Simulation and Architecture Defense

Project Goal

Integrate delivery, infrastructure, identity, observability, and recovery into one enterprise platform. Execute a multi-failure exercise and defend the architecture.

Engineering Challenge

Lead a realistic incident, quantify blast radius, recover safely, and justify architecture decisions.

Enterprise Platform Architecture (End-to-End)

1. Delivery and Applications

- Source Code (Git)
- CI/CD Pipeline (Build / Test / Deploy)
- Container Registry (Images)
- Applications (Microservices)

2. Infrastructure and Compute

- Kubernetes (Multi-AZ / Multi-Region)
- Compute Nodes (Auto Scaling)
- Ingress & Load Balancing
- Storage (Block / Object)

3. Identity and Security

- Identity Provider (IAM / SSO / RBAC)
- Secrets Management (Vault / KMS)
- Policy & Compliance (OPA / Policies)
- Access Reviews (Users / Roles)

4. Observability and Monitoring

- Metrics (Prometheus)
- Logs (Loki / ELK)
- Traces (OpenTelemetry)
- Alerting (Alertmanager)
- Dashboards (Grafana)

5. Resilience and Recovery

- Automated Recovery (Runbooks)
- Backup & Restore (Automated)
- Disaster Recovery (Multi-Region)
- Chaos / Failure Simulations
- Incident Response (Detection → Recovery)

Key Components

- End-to-end enterprise platform (applications, data, security, observability)
- Multi-failure scenario design and execution
- Identity, access, and secrets security
- CI/CD with automated deployment and rollback
- Observability (metrics, logs, traces, alerting)
- Incident detection, response, and recovery
- Blast radius analysis and impact quantification
- Architecture defense and decision justification
- Post-incident review and documentation
- Automation and runbook validation

Example Multi-Failure Exercise Flow

1. Inject Failures

- Simulate multiple failures (e.g., node loss, DB failure, network latency, service timeout, DNS issue)

2. Detect and Analyze

- Identify impact and blast radius
- Analyze metrics, logs and traces
- Prioritize recovery actions

3. Execute Recovery

- Trigger automated runbooks
- Perform manual actions if needed
- Restore services safely
- Validate data integrity

4. Validate and Stabilize

- Confirm full service recovery
- Run functional and performance tests
- Ensure no hidden failures
- Stabilize and monitor

5. Defend Architecture

- Present timeline and impact
- Justify architectural decisions
- Explain trade-offs and improvements
- Propose additional resilience measures

Success Criteria

- [ ] All simulated failures are detected and handled
- [ ] Services recover within defined SLOs
- [ ] No data loss or corruption
- [ ] Blast radius is accurately quantified
- [ ] Architecture remains resilient under stress
- [ ] Automated and manual recovery procedures are validated
- [ ] Clear documentation and post-incident analysis
- [ ] Architecture decisions are justified with evidence
- [ ] Lessons learned are captured and applied
- [ ] Platform demonstrates enterprise-grade reliability

Failure Scenarios to Test

- [ ] Simultaneous node and database failure
- [ ] Network partition between services
- [ ] DNS outage or misconfiguration
- [ ] Identity provider (SSO/IAM) unavailable
- [ ] Secrets manager access failure
- [ ] Storage failure or data corruption
- [ ] High traffic spike with cascading service failures
- [ ] Regional outage (multi-AZ / multi-region)
- [ ] CI/CD pipeline or deployment failure during incident
- [ ] Monitoring or alerting system failure